Coding agent benchmark

Six harness/model pairs. Twelve curated tasks. Seventy-two scored attempts.

Objective

Measure task completion, execution time and cost across six coding agent configurations. Use the results to compare tools for the selected workload.

Task selection

The CPU-only subset contains six easier tasks with expert estimates of 0.75–2 hours and six harder tasks estimated at 4–8 hours. Every pair receives the same tasks.

Twelve tasks keep the local study tractable across six configurations. The selection covers security, routing geometry, music constraints, build artifacts, cryptanalysis, payments, storage, live systems, graph performance and UI behavior.

Study scope

The unit of comparison is the configured harness/model pair. The dataset is curated, with one attempt per pair and task. Scores describe completion on these twelve tasks.

Methodology

Execution and verification

Harbor starts each task environment and installs the pinned harness. The runner supplies the task and model, requests medium reasoning effort, and executes one trial at a time. The agent timeout multiplier is 0.25; an eight-hour task receives a two-hour agent limit.

A separate verifier checks artifacts and behavior after execution. Reward 1 means resolved. Each pair has twelve scored attempts. Runtime failures and provider refusals count as unresolved.

Configuration and accounting

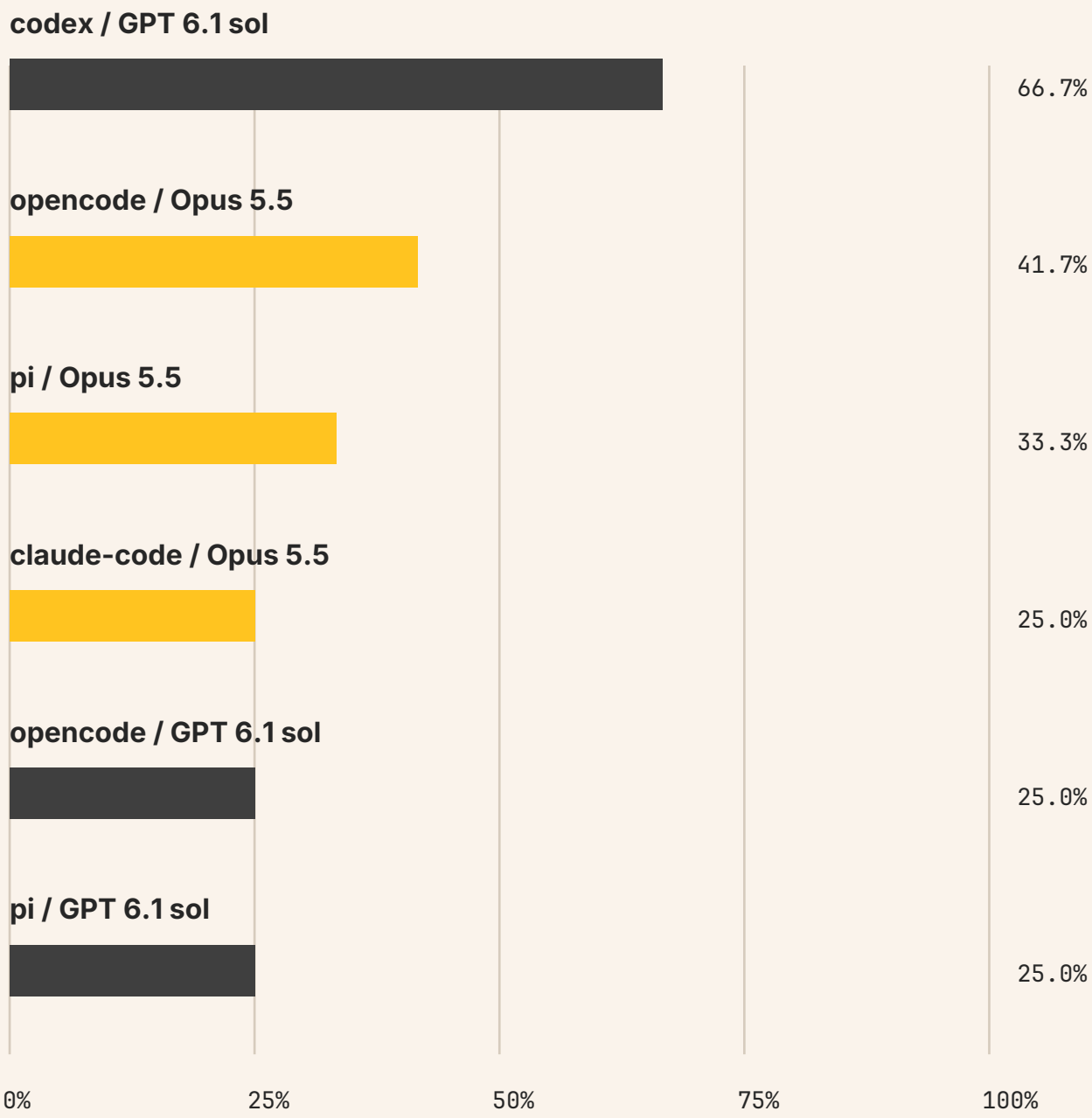
OpenCode on Opus uses a 128,000-token output cap. Other configurations retain their recorded defaults. Cost uses agent-reported values where available, otherwise pinned API prices. Time covers agent execution, excluding setup and verification.

Evidence and exclusions

GPT and Claude Code results use sealed r3. The later pi and OpenCode Opus trials have hash-locked evidence from a dirty checkout, outside the clean-commit seal. Two OpenCode setup failures are retained and replaced: html-js-filter scored 0 and photonic-waveguide-routing scored 1. The earlier 32k Opus smoke is excluded.

Task resolution

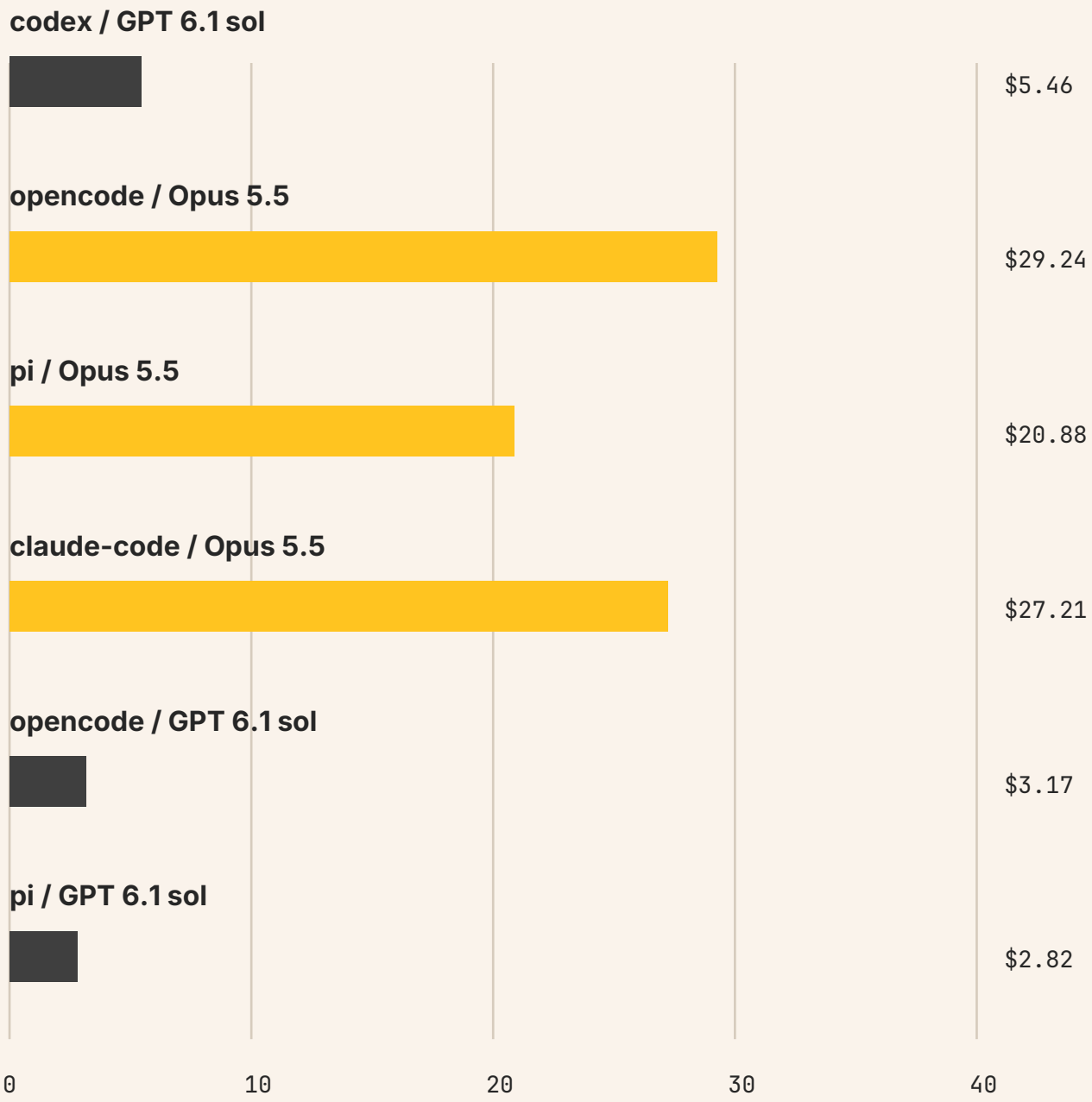
Resolved tasks / 12 (%). Amber: Opus 5.5. Graphite: GPT 6.1 sol.



Codex / GPT resolves 8 tasks; OpenCode / Opus resolves 5; pi / Opus resolves 4. The other pairs resolve 3 each.

Cost ($)

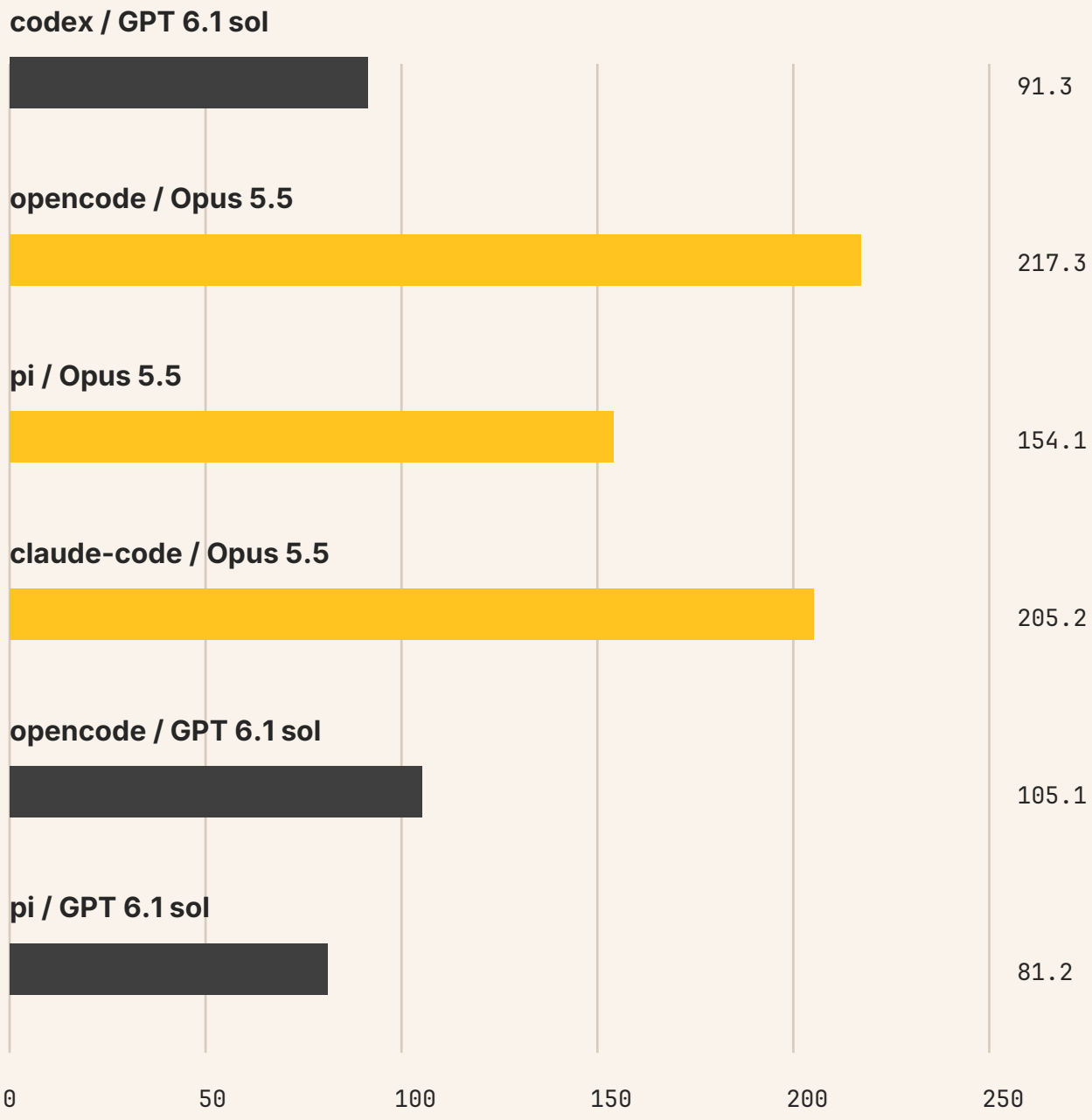
Total cost ($) across twelve scored attempts per pair.



Pi / GPT has the lowest measured cost at $2.82. Codex / GPT resolves the most tasks at $5.46.

Execution time

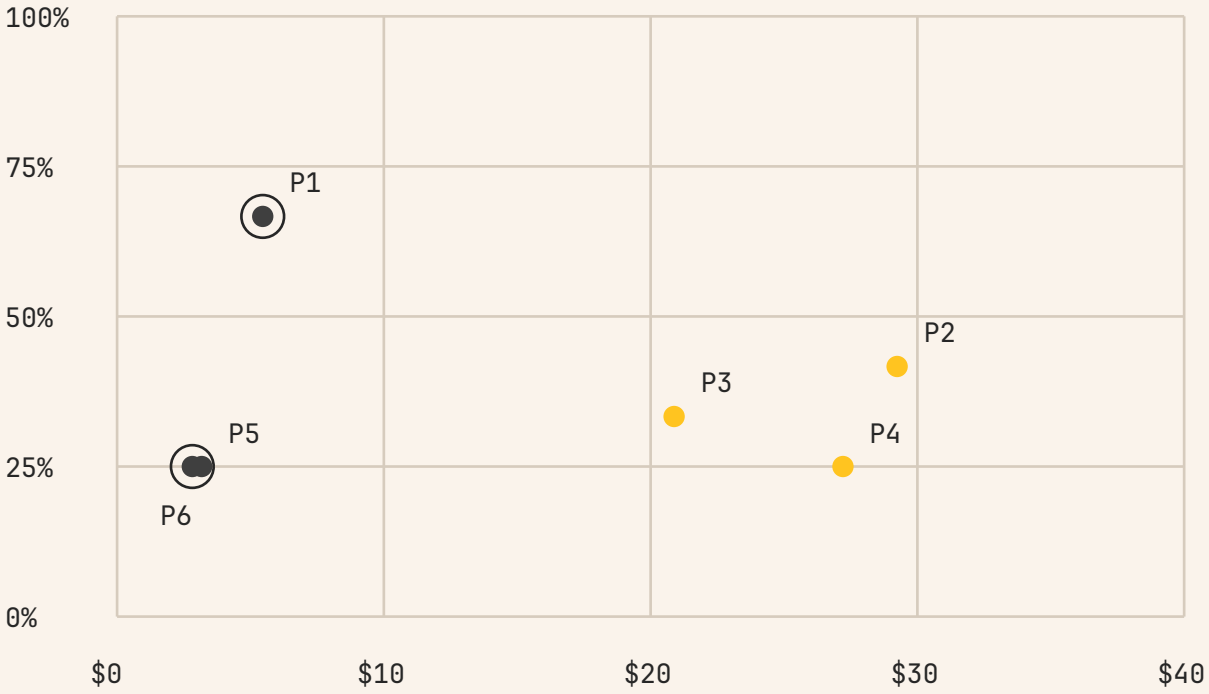
Total agent execution time (minutes) across twelve scored attempts per pair.



Totals exclude environment setup, verifier execution and excluded attempts.

Cost and resolution

Resolution rate (%) against cost ($). Rings mark non-dominated pairs.



P1 codex / GPT 6.1 sol

P2 opencode / Opus 5.5

P3 pi / Opus 5.5

P4 claude-code / Opus 5.5

P5 opencode / GPT 6.1 sol

P6 pi / GPT 6.1 sol

The observed frontier contains pi / GPT and Codex / GPT. A non-dominated pair has no alternative that costs no more and resolves at least as many tasks, with one strict improvement.

Results ledger

P1–P6 identify the same configurations in the charts and task matrix.

P1 codex / GPT 6.1 sol

8/12 passed	91.3 min	$5.46	$0.68/pass

P2 opencode / Opus 5.5

5/12 passed	217.3 min	$29.24	$5.85/pass

P3 pi / Opus 5.5

4/12 passed	154.1 min	$20.88	$5.22/pass

P4 claude-code / Opus 5.5

3/12 passed	205.2 min	$27.21	$9.07/pass

P5 opencode / GPT 6.1 sol

3/12 passed	105.1 min	$3.17	$1.06/pass

P6 pi / GPT 6.1 sol

3/12 passed	81.2 min	$2.82	$0.94/pass

Harness versions: Codex 0.163.0; Claude Code 2.1.302; OpenCode 1.19.3; pi 1.3.0. Opus authentication plugins: OpenCode 1.8.6; pi 3.4.3.

Task-level outcomes

1 = resolved. 0 = unresolved. * = runtime exception or provider refusal.

	P1	P2	P3	P4	P5	P6
bun-sourcemap-leak	0	0	0	0	0	0
html-js-filter	1	0	0	0	1	0
interleaved-vigenere	0	1	1	0*	0	0
kv-live-surgery	1	0*	0	0	0	0
music-harmony	0	0	0	0	0	0
mvcc-lsm-compaction	1	0	0	0	0	0
payments-pipeline-fix	1	1	1	1	0	1
photonic-waveguide-routing	1	1	1	1	1	1
react-lead-form	0	1	1	1	0	0
session-window-debug	1	0	0	0	0	0
vf2-speedup-networkx	1	1	0	0	0*	0
wal-recovery-ordering	1	0	0	0	1	1

Provider refusals occurred for Claude Code on interleaved-vigenere and OpenCode Opus on kv-live-surgery. OpenCode GPT hung on vf2-speedup-networkx; its cause remains unresolved.

Interpretation

Completion and cost

Codex / GPT leads completion with 8/12 resolved tasks at $5.46. Pi / GPT has the lowest total cost at $2.82 and resolves 3/12. These pairs define the observed cost–resolution frontier.

Within-harness differences

OpenCode resolves 5/12 tasks with Opus and 3/12 with GPT. Pi resolves 4/12 with Opus and 3/12 with GPT. The higher Opus scores come with higher costs. These differences include model, plugin and budget effects.

Task sensitivity

One task changes the resolution rate by 8.3 percentage points. No configuration resolves bun-sourcemap-leak or music-harmony. Repeated runs can distinguish stable failure patterns from run-to-run variation.

Use of the results

For this workload, Codex / GPT offers the highest completion rate, while pi / GPT offers the lowest measured cost. Tool selection can use these two reference points alongside the task-level results.

References

Study records

Task selection: tasks.toml. Scored outcomes and aggregate metrics: results/opus-comparison/data.json. Evidence hashes: results/opus-comparison/manifest.json.

Presentation assets and PDF hashes: results/opus-comparison-branded/manifest.json. The report uses the same 72 scored attempts as the source comparison.

Evaluation methodology

Anthropic — SWE-bench engineering report. Reference for evaluating a model together with its harness.

https://www.anthropic.com/engineering/swe-bench-sonnet

Terminal-Bench — leaderboard presentation of task resolution and uncertainty. The fetched page at the link below displayed Terminal-Bench 4.0.

https://www.tbench.ai/leaderboard/terminal-bench/2.0

Brand assets

Blockether — typeface, symbol and wordmark geometry from https://blockether.com/. The source font and measured geometry are retained with the presentation assets.

Further work

01 / Repeat the trials

Run multiple attempts for each pair and task. Report variation in completion, cost and time, then estimate uncertainty from the repeated measurements.

02 / Match the budgets

Align output caps, reasoning settings and time limits where the harnesses permit. Record remaining configuration differences alongside each result.

03 / Expand task coverage

Add tasks across difficulty levels and domains. Define the selection procedure before execution and retain task-level outcomes for each configuration.

04 / Resolve failure mechanisms

Reproduce the OpenCode GPT hang on vf2-speedup-networkx. Inspect verifier artifacts for the shared failures on bun-sourcemap-leak and music-harmony.

05 / Standardize the next run

Run all configurations from a clean, pinned checkout in one campaign. Use a common cost ledger and apply the standard evidence seal to every trial.